Week 2 Report - Data Wrangling and Exploration

Report coverage, missingness, duplicates, implausible values, and cleaning decisions. For joins, report unmatched keys and row counts before and after joining. Include both code and explanatory text describing your findings and assessment in your submitted knitted PDF or HTML.

Census Data

Overview

Our team was looking to capture median household income for a California zipcodes from 2021 - 2024. We plan on expanding this coverage to contain 2025 data once the US Census releases those numbers in the coming week which will enable us to have the same years of analysis for household income and insurance data. As seen in *ACS_Import_Clean.ipynb,* the data was scrapped via the *pyttidcensus* package with an API key so we could automatically scrape the ZCTA, polygons of the area, and the income data for each year. Afterwards, the results were filtered down to contain only California ZCTA as seen in the graphic below.

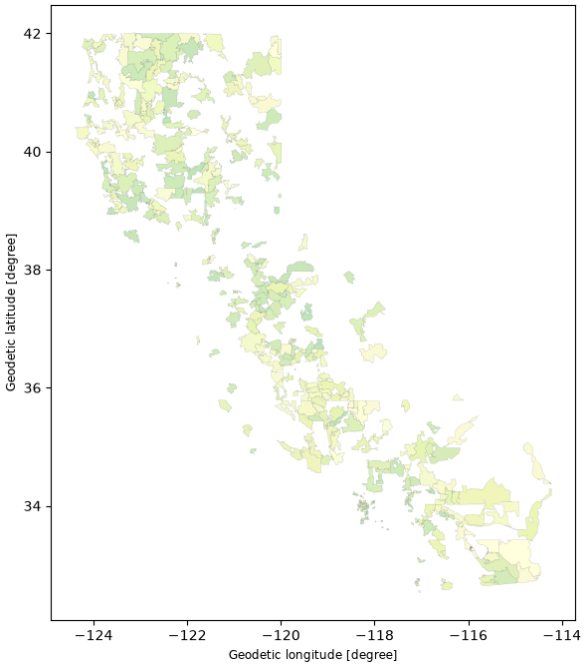


Figure 1: Visualization of ZCTA in California for 2021 US Census data

Data Wrangling

The Census data scrapped for 2021 - 2024 was very strong upon further assessment, as seen in our *CSV_Data_Quality_Audit.ipynb* script. Upon converting the ZCTA to zip codes for future assessment, no mismatches, duplicates, or implausible values appeared in the dataset thus indicating our team was engaging with a clean dataset from the onset.

An example of our ZCTA, as seen below, indicated there were 0 missing values for all years in the assessed census data.

```
census_2021['NAME'].isna().sum()

# no NA ZCTA values

_
Results
np.int64(0)
```

```
# repeat checks for each year

dfs = [census_2022, census_2023, census_2024]

missing = 0

for df in dfs:
   missing = missing + df['NAME'].isna().sum()

print(missing)

# no missing ZCTAs
```

Upon merging the datasets, an additional check was done to ensure the translation of the ZCTA to zipcode was done correctly to ensure no issues arose when creating the master *Income_2021_2024.csv.*

```
# join with zcta to zip code

cfp_data = cfp_data.rename(columns={
   'zip' : 'zip_code'
})

cfp_zips = pd.merge(
```

```
    cfp_data,
    zcta_to_zip,
    on='zip_code',
    how='left',
    indicator=True
)

cfp_zips['_merge'].value_counts()
--
Results
_merge
both          1682
left_only        0
right_only       0
Name: count, dtype: int64
```

Conclusion and Next steps

Connecting US Census data to insurance data to begin more comprehensive trends assessment.

California Fair Plan Property Insurance PDF Scrape

Overview

Three pdfs were scraped containing California Fair Plan Property Insurance information on the following: (1) general plan data; (2) residential risk exposure per zipcode; (3) commercial risk exposure per zipcode.

Data Wrangling

The pdflumber package was utilized to combine each master PDF of 30+ pages into one centralized data frame as seen for residential risk below. AI assistance was required in order to complete this task given this was outside the purview of lessons in the first half of the course which focused on web based scrapping.

```
table_settings = {
   'vertical_strategy': 'text',
   'horizontal_strategy': 'text',
}
rows = []  # reset each run so reruns don't duplicate rows

#help provided with some AI, particularly with zip_cell check to get unified, scraped
dataframe
with pdfplumber.open(residential_risk) as pdf:
   for page in pdf.pages:
       for table in page.extract_tables(table_settings):
           for row in table:
               zip_cell = (row[0] or '').strip()
               if re.fullmatch(r'\d{5}', zip_cell):
                   # clean each cell: None -> '', drop spaces, commas, and %
                   clean_row = [(cell or '').replace(' ', '').replace(',',
'').replace('%', '')
                                for cell in row]
                   rows.append(clean_row)

#need to add 4 $ columns to align with the scraping of the pdf
cols = ['zip',
       'growth_2025', '$', 'exposure_2025',
       'growth_2024', '$', 'exposure_2024',
       'growth_2023', '$', 'exposure_2023',
       'growth_2022', '$', 'exposure_2022',
       'growth_2021', 'exposure_2021']
residential_risk = pd.DataFrame(rows, columns=cols)
```

Upon creating the core dataframe, the following wrangling was conducted:

1. Deleting the $ columns which continued to appear for the commercial and residential risk assessment pdfs
2. Converting the growth columns into the proper decimal format as a percentage
3. Updating each column type accordingly

In order to ensure the scrapping was successful, a summation of the total exposure and plans was created and manually checked with the given values on the pdf. The matching of the results to the given value on the pdf signaled that the dataframe was clean.

```
#check to see if numbers from pull are aligned with pdf
# Plans from pdf
# 2025 exposure = $645,115,692,650
# 2024 exposure = $431,066,598,171
# 2023 exposure = $270,510,184,565
total_2025_exposure = residential_risk_clean['exposure_2025'].sum()
total_2024_exposure = residential_risk_clean['exposure_2024'].sum()
total_2023_exposure = residential_risk_clean['exposure_2023'].sum()

print('2025 exposure:',total_2025_exposure)
print('2024 exposure:',total_2024_exposure)
print('2023 exposure:',total_2023_exposure)

_
Results
2025 exposure: 645115692650
2024 exposure: 431066598171
2023 exposure: 270510184565
```

Next Step

The one scrape which needs to be troubleshooted further is the commercial risk exposure given the mismatch in the 2024 exposure from the pdf as opposed to the value given in the final dataframe as seen below.

```
#check to see if numbers from pull are aligned with pdf
# Plans from pdf
# 2025 exposure = $48,801,762,839
# 2024 exposure = $26,876,155,001
# 2023 exposure = $13,445,162,119
# 2022 exposure = $8,116,443,171
```

```
total_2025_exposure = commercial_risk_clean['exposure_2025'].sum()
total_2024_exposure = commercial_risk_clean['exposure_2024'].sum()
total_2023_exposure = commercial_risk_clean['exposure_2023'].sum()
total_2022_exposure = commercial_risk_clean['exposure_2022'].sum()

print('2025 exposure:',total_2025_exposure)
print('2024 exposure:',total_2024_exposure)
print('2023 exposure:',total_2023_exposure)
print('2023 exposure:',total_2022_exposure)

_
2025 exposure: 48801762839
2024 exposure: 27046604193
2023 exposure: 13445162119
2023 exposure: 8116443171
```

Cal Fire Data

Overview

California Fire Perimeters from the 1950 onwards was downloaded from the California Open Data portal. This is the premier historic wildfire data set for the state that explains the cause of the fire and its burned acreage.

Data Wrangling

The primary focus for this week was to ensure data was accessible for the team and could be visualized in our coding environments. The primary blocker as of now is deciding on a methodology on how to assign the wildfire polygons to associated zipcodes. The team is exploring for wildfires the following options:

1. Assigning each wildfire to one zipcode
2. Assigning the wildfire to each zipcode that it touches which would require dissecting up the wildfire in accordance with the zipcode polygons. We'd then need to consider a weighting system of the wildfire for each zipcode as there is expected to be varying effects of each wildfire in each zipcode.

Key blocker and next step: Finding geopandas or arcpy for zipcode attribution

No package or coding solution has emerged that can connect the zipcode polygons to the wildfire polygons to enable wildfire attribution.

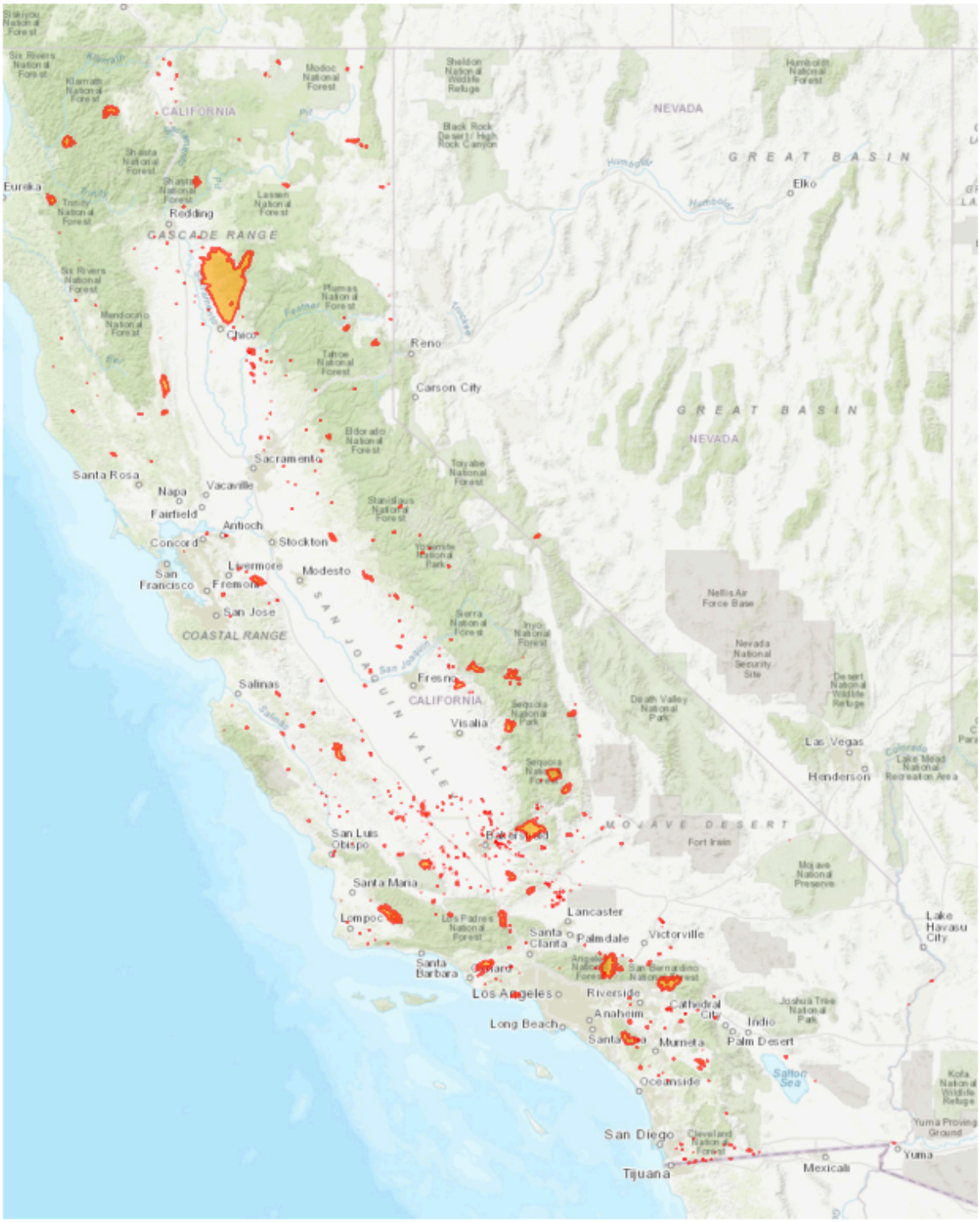


Figure 2: Visualization test of California wildfires in 2024.